JAVA P1

Riassunto completo + esempi + trick di memoria + piano fino al 27 + esercizi + correzioni

Questa dispensa è costruita sui 35 fogli del materiale del corso. L'obiettivo non è solo leggere, ma saper spiegare e scrivere codice senza guardare gli appunti.

> TRICK: Usa il PDF in 3 passaggi: 1) leggi una sezione; 2) chiudila e ripetila a voce; 3) scrivi il codice da zero. Se sai solo riconoscere il codice quando lo vedi, non lo hai ancora memorizzato.

Regola anti-soluzioni: la parte finale contiene tutte le correzioni. Non leggerla prima di aver completato esercizi e domande.

1. Piano per imparare tutto prima del 27

Dal 19 al 27 hai circa una settimana piena. Il metodo più efficace qui è alternare teoria, richiamo attivo e codice. Ogni sessione può durare 60-90 minuti; se hai più tempo, ripeti il blocco pratico.

Giorno	Obiettivo
19 ottobre	Java: caratteristiche, JVM, JDK/JRE, compilato-interpretato-ibrido, IDE. Ripeti a voce senza appunti.
20 ottobre	Struttura del programma, main, println/print, librerie, package, import, Scanner.
21 ottobre	Tipi di dato, variabili, costanti, operatori, caratteri speciali, casting, pre/post incremento.
22 ottobre	Math e Math.random(). Scrivi almeno 8 piccoli programmi senza copiare.
23 ottobre	if / else if / else, operatori logici, ternario, if annidato, switch.
24 ottobre	while, do-while, for, foreach. Trasforma lo stesso esercizio usando cicli diversi.
25 ottobre	array, matrici, chiamate di metodi, eccezioni, BufferedReader/InputStreamReader.
26 ottobre	Simulazione completa: domande teoriche + esercizi del PDF. Correggi solo alla fine e riscrivi gli errori.
27 ottobre	Ripasso leggero: formule/trick, sintassi essenziale e 3 esercizi brevi. Niente studio caotico dell'ultimo minuto.

Metodo di memorizzazione

10-3-1: 10 minuti di studio, 3 minuti a libro chiuso per spiegare, 1 mini-esempio scritto da zero. Alla fine della giornata fai un richiamo di 10 minuti degli argomenti del giorno precedente.

Semaforo: verde = lo spiego e lo programmo; giallo = lo riconosco ma mi blocco; rosso = non ricordo. Il 26 devi lavorare soprattutto sui gialli e rossi.

2. Java: caratteristiche e funzionamento

Java viene presentato nel materiale come linguaggio nato nel 1995, con sintassi simile a C/C++, robusto, orientato agli oggetti e soprattutto portabile. L'indipendenza dalla piattaforma dipende dalla JVM: il sorgente .java viene compilato in bytecode .class, poi la JVM interpreta/esegue quel bytecode sul sistema operativo.

> TRICK: .java → .class → JVM → esecuzione. Pensa a una staffetta in 4 tappe. Se ricordi questa freccia, ricordi perché Java è considerato ibrido e portabile.

Compilato: il sorgente viene tradotto in codice macchina prima dell'esecuzione. Interpretato: un interprete esegue le istruzioni durante l'esecuzione. Ibrido: prima compila in codice intermedio, poi una macchina virtuale lo esegue. Nel corso Java è l'esempio principale di linguaggio ibrido.

JDK, JRE, JVM e IDE

JDK è il kit di sviluppo e nel materiale include compilatore javac, JRE e JVM. Gli IDE/ambienti con GUI citati sono IntelliJ, Eclipse, NetBeans e VS Code. Da prompt si può compilare manualmente.

> TRICK: Immagina scatole: JDK = scatola grande per sviluppare; dentro trovi gli strumenti necessari per compilare/eseguire. JVM = la macchina che fa girare il bytecode.

3. Classi, librerie, import e primo programma

Java è orientato agli oggetti: gli oggetti nascono da classi. Le classi hanno attributi e metodi. Le librerie raccolgono funzionalità già pronte. Per richiamare classi esterne si usa import; il materiale cita, tra gli altri, java.util.Scanner, java.lang.* e java.io.*.

Scheletro fondamentale

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

public è un modificatore di accesso; static permette di richiamare il metodo senza creare un oggetto; void indica che il metodo non restituisce un valore; String[] args contiene eventuali argomenti da riga di comando. System.out.println stampa e va a capo.

> TRICK: Memorizza il main come una frase ritmica: public static void main String args. Poi aggiungi parentesi e parentesi graffe. Scrivilo 5 volte senza copiare.

4. Input con Scanner

```
import java.util.Scanner;

public class Esempio {
    public static void main(String[] args) {
        Scanner s1 = new Scanner(System.in);

        System.out.print("Inserisci un numero: ");
        int numero = s1.nextInt();

        System.out.println("Hai scritto: " + numero);
    }
}
```

Scanner s1 = new Scanner(System.in); crea l'oggetto Scanner. I metodi principali del materiale sono: next() per una parola, nextLine() per una riga, nextInt(), nextLong(), nextFloat(), nextDouble() e nextBoolean().

> TRICK: Il nome dopo next ti dice cosa entra: nextInt → int, nextDouble → double. nextLine è l'eccezione facile da ricordare: prende tutta la LINEA.

5. Output, variabili, costanti e operatori

println stampa e va a capo; print resta sulla stessa riga. Tra i tipi primitivi mostrati: byte, short, int, long, float, double, boolean e char. Una costante si dichiara con final, per esempio final double PI = 3.1415;.

```
int eta = 20;
double altezza = 1.75;
boolean maggiorenne = true;
char iniziale = 'A';
final double PI = 3.1415;
```

Operatori aritmetici: + - * / %. Incremento/decremento: ++ --. Assegnazioni compatte: += -= *= /=. Confronti: == != > < >= <=. Logici: && (AND), || (OR). Il materiale distingue anche & e | come operatori binari.

> TRICK: = mette un valore dentro una variabile; == domanda se due valori sono uguali. Uno assegna, due confrontano.

Caratteri speciali

Tra quelli elencati: \n nuova riga, \t tabulazione, \\ backslash e sequenze per inserire apici nelle stringhe.

```
System.out.println("Nome:\tMario\nEtà:\t20");
```

6. Pre/post incremento e casting

```
int x = 3;
int y = (++x) + 1;   // prima x diventa 4, poi si usa: y = 5

int a = 3;
int b = (a++) + 1;   // prima si usa 3: b = 4, poi a diventa 4
```

> TRICK: ++ prima = aumenta PRIMA. ++ dopo = usa il valore, POI aumenta.

Il casting converte un tipo in un altro. Esempio tipico: int b = (int) a;. Se trasformi un numero decimale in int, la parte decimale viene persa.

```
double prezzo = 7.9;
int intero = (int) prezzo;   // 7
```

7. Classe Math e numeri casuali

Nel materiale compaiono Math.abs, Math.max, Math.min, Math.sqrt e Math.random.

```
int n = (int)(Math.random() * (max - min + 1)) + min;
```

> TRICK: Formula random: ampiezza × random + minimo. L'ampiezza inclusiva è max - min + 1.

```
int dado = (int)(Math.random() * 6) + 1;       // 1..6
int n = (int)(Math.random() * 146) + 55;       // 55..200
```

8. Selezione: if, else, ternario e switch

```
if (x > y) {
    System.out.println("x maggiore");
} else if (x < y) {
    System.out.println("y maggiore");
```

```
} else {
    System.out.println("uguali");
}
```

Un if annidato è un if dentro un altro if. Con gli operatori logici puoi unire condizioni, per esempio eta >= 18 && eta < 65.

Operatore ternario: condizione ? vero : falso

```
String risultato = (eta >= 18) ? "Maggiorenne" : "Minorenne";
```

> TRICK: Ternario = una domanda con due uscite: ? se vero : se falso.

```
switch (giorno) {
    case "Lunedi":
        System.out.println("Inizio settimana");
        break;
    case "Sabato":
    case "Domenica":
        System.out.println("Weekend");
        break;
    default:
        System.out.println("Giorno normale");
}
```

> TRICK: Nello switch pensa a un casellario: case sceglie il cassetto, break chiude il cassetto, default è il piano B.

9. Cicli: while, do-while, for e foreach

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```

while: prima controlla la condizione, poi esegue. Quindi può eseguire zero volte.

```
int i = 0;
do {
    System.out.println(i);
    i++;
} while (i < 5);
```

do-while: prima esegue il blocco, poi controlla. Quindi esegue almeno una volta.

> TRICK: WHILE = guardia alla porta: controlla prima di entrare. DO-WHILE = sei già dentro: fai almeno un giro.

```
for (int i = 0; i < numeri.length; i++) {
    System.out.println(numeri[i]);
}

for (int numero : numeri) {
    System.out.println(numero);
}
```

for usa l'indice; foreach usa direttamente l'elemento e serve a scorrere tutti gli elementi del vettore senza gestire l'indice.

10. Array, matrici e new

```
String[] auto = {"Volvo", "BMW", "Ford", "Mazda"};
System.out.println(auto[0]);

int[] vet = new int[5];
```

```
int[][] matrice = {
    {1, 4, 2},
    {3, 6, 8}
};
System.out.println(matrice[0][1]); // 4
```

Gli array sono vettori monodimensionali; le matrici usano due indici. new serve a creare una nuova struttura in memoria, come un oggetto o un vettore.

> TRICK: Gli indici partono da 0. Primo elemento = [0], secondo = [1]. In una matrice pensa [riga][colonna].

11. Metodi / chiamate di funzione

Nel materiale si distingue tra chiamare un metodo della stessa classe e chiamare un metodo di un'altra classe. Nella stessa classe il metodo può essere richiamato direttamente quando la struttura lo consente; in un'altra classe, nell'esempio mostrato, si crea un oggetto e poi si usa la dot notation.

```
class A {
    void metodo() {
        System.out.println("Ciao");
    }
}

class B {
    public static void main(String[] args) {
        A oggetto = new A();
        oggetto.metodo();
    }
}
```

> TRICK: Dot notation = chi.faCosa(). Prima l'oggetto, poi il punto, poi il metodo.

12. Eccezioni: try, catch, finally

Le eccezioni sono eventi anomali durante l'esecuzione. try contiene il codice rischioso; catch intercetta/gestisce l'errore; finally viene eseguito comunque ed è facoltativo.

```
try {
    int risultato = 10 / 0;
} catch (Exception e) {
    System.out.println("Errore");
} finally {
    System.out.println("Fine del programma");
}
```

> TRICK: TRY = provo, CATCH = acchiappo l'errore, FINALLY = alla fine lo faccio comunque.

13. InputStreamReader e BufferedReader

Il materiale spiega che InputStreamReader converte byte in caratteri e legge un carattere alla volta; per leggere comodamente righe intere si aggiunge BufferedReader, che permette di usare readLine(). BufferedReader è indicato come conveniente per grossi flussi di testo.

```
InputStreamReader isr = new InputStreamReader(System.in);
BufferedReader br = new BufferedReader(isr);

System.out.print("Inserisci il tuo nome: ");
String nome = br.readLine();
System.out.println("Ciao " + nome);
```

14. ESERCIZI - NON GUARDARE LE CORREZIONI

Fai prima tutti gli esercizi e le domande. Segna con una stella quelli in cui hai dubbi. Solo dopo passa alla sezione CORREZIONI.

A. Domande teoriche

1. Perché Java è indipendente dalla piattaforma?
2. Spiega il percorso da un file .java fino all'esecuzione.
3. Qual è la differenza tra linguaggio compilato, interpretato e ibrido?
4. Che ruolo hanno JDK, javac e JVM nel materiale?
5. Che differenza c'è tra classe e oggetto?
6. A cosa serve import? Cos'è un package?
7. Spiega public, static, void e String[] args nel main.
8. Differenza tra print e println.
9. Differenza tra next() e nextLine().
10. Che differenza c'è tra = e ==?
11. A cosa servono && e ||?
12. Differenza tra ++x e x++ durante un'assegnazione.
13. Cos'è il casting?
14. Scrivi a memoria la formula per un casuale intero tra min e max inclusi.
15. Quando useresti if/else e quando switch?
16. Differenza fondamentale tra while e do-while.
17. Differenza tra for e foreach.
18. Da quale indice parte un array?
19. A cosa servono try, catch e finally?
20. Perché si usa BufferedReader insieme a InputStreamReader nel materiale?

B. Prevedi l'output

B1

```
int x = 3;
int y = (++x) + 2;
System.out.println(x);
System.out.println(y);
```

B2

```
int x = 3;
int y = (x++) + 2;
System.out.println(x);
System.out.println(y);
```

B3

```
int n = 7;
if (n % 2 == 0) {
    System.out.println("A");
} else {
    System.out.println("B");
}
```

B4

```
for (int i = 1; i <= 3; i++) {
    System.out.print(i + " ");
```

```
}
```

C. Trova e correggi l'errore

C1

```
public class Test {
    public static int main(String[] args) {
        System.out.println("Ciao")
    }
}
```

C2

```
Scanner s = new Scanner(System.in);
int eta = s.nextLine();
```

C3

```
int x = 5;
if (x = 5) {
    System.out.println("uguale");
}
```

C4

```
int[] numeri = {10, 20, 30};
System.out.println(numeri[3]);
```

D. Esercizi di programmazione

D1. Chiedi nome ed età con Scanner e stampa: «Ciao NOME, hai ETA anni».

D2. Leggi due interi e stampa il maggiore; se sono uguali scrivi «uguali».

D3. Leggi un numero e indica se è pari o dispari usando %.

D4. Genera un numero casuale da 1 a 10 e stampalo.

D5. Genera un numero casuale da 55 a 200 senza copiare la formula.

D6. Stampa i numeri da 1 a 10 con un for.

D7. Stampa i numeri da 10 a 1 con un while.

D8. Crea un array {5, 10, 15, 20} e stampalo prima con for e poi con foreach.

D9. Crea una matrice 2x3 e stampa l'elemento della prima riga, seconda colonna.

D10. Leggi un voto intero: <6 «insufficiente», da 6 a 7 «sufficiente», da 8 a 9 «buono», 10 «ottimo».

D11. Usa switch per stampare un messaggio diverso per 1, 2, 3 e un default per gli altri valori.

D12. Scrivi un try-catch che gestisca un errore di divisione e un finally che stampi «fine».

D13. BONUS: chiedi 5 numeri all'utente, salvali in un array e stampa il maggiore.

15. STOP - CORREZIONI

> ⚠ **NON LEGGERE DA QUI IN POI SE NON HAI FINITO. Se guardi la soluzione troppo presto alleni il riconoscimento, non il richiamo dalla memoria.**

Correzioni domande teoriche - risposta essenziale

1. Perché il sorgente viene compilato in bytecode e la JVM lo esegue sulla piattaforma disponibile.

2. .java → compilazione con javac → .class/bytecode → JVM → esecuzione/runtime.

3. Compilato: traduzione prima dell'esecuzione; interpretato: esecuzione tramite interprete; ibrido: compilazione in codice intermedio + interpretazione/macchina virtuale.

4. JDK: kit di sviluppo; javac: compilatore Java; JVM: esegue/interpreta il bytecode.

5. La classe è il modello/struttura; l'oggetto è un'istanza creata a partire dalla classe.

6. import richiama classi/librerie esterne; package raggruppa classi.

7. public: accessibilità; static: richiamabile senza creare un oggetto; void: nessun valore restituito; String[] args: argomenti da riga di comando.

8. println va a capo, print no.

9. next legge una parola/token; nextLine legge l'intera riga.

10. = assegna; == confronta l'uguaglianza.

11. && richiede entrambe le condizioni vere; || ne richiede almeno una vera.

12. ++x incrementa prima dell'uso; x++ usa prima il vecchio valore e incrementa dopo.

13. Conversione di un dato da un tipo a un altro.

14. (int)(Math.random() * (max - min + 1)) + min

15. if/else è adatto a condizioni generiche/intervalli; switch a selezioni multiple su casi distinti.

16. while controlla prima; do-while esegue prima e quindi almeno una volta.

17. for gestisce tipicamente l'indice; foreach prende direttamente ogni elemento.

18. 0.

19. try contiene codice che può generare eccezioni; catch le gestisce; finally viene eseguito comunque ed è facoltativo.

20. InputStreamReader converte byte in caratteri; BufferedReader accumula i caratteri e consente di leggere righe con readLine().

Output

B1: x = 4, y = 6. Il preincremento cambia x prima del calcolo.

B2: x = 4, y = 5. Il postincremento usa prima 3 e poi porta x a 4.

B3: B, perché 7 non è divisibile per 2.

B4: 1 2 3

Errori

C1 - main deve essere void e mancava ;

```
public class Test {
    public static void main(String[] args) {
        System.out.println("Ciao");
    }
}
```

C2 - nextLine restituisce una String, per un int usa nextInt()

```
Scanner s = new Scanner(System.in);
int eta = s.nextInt();
```

C3 - confronto con ==

```
int x = 5;
if (x == 5) {
    System.out.println("uguale");
}
```

C4 - gli indici validi sono 0, 1, 2

```
int[] numeri = {10, 20, 30};
System.out.println(numeri[2]);
```

Soluzioni esercizi di programmazione

D1

```
import java.util.Scanner;

public class D1 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Nome: ");
        String nome = s.nextLine();
        System.out.print("Eta: ");
        int eta = s.nextInt();
        System.out.println("Ciao " + nome + ", hai " + eta + " anni");
    }
}
```

D2

```
int a = s.nextInt();
int b = s.nextInt();

if (a > b) {
    System.out.println(a);
} else if (b > a) {
    System.out.println(b);
} else {
    System.out.println("uguali");
}
```

D3

```
int n = s.nextInt();
if (n % 2 == 0) {
    System.out.println("pari");
} else {
    System.out.println("dispari");
}
```

D4

```
int n = (int)(Math.random() * 10) + 1;
System.out.println(n);
```

D5

```
int n = (int)(Math.random() * 146) + 55;
System.out.println(n);
```

D6

```
for (int i = 1; i <= 10; i++) {
    System.out.println(i);
}
```

D7

```
int i = 10;
while (i >= 1) {
    System.out.println(i);
    i--;
}
```

D8

```
int[] numeri = {5, 10, 15, 20};

for (int i = 0; i < numeri.length; i++) {
    System.out.println(numeri[i]);
}

for (int numero : numeri) {
    System.out.println(numero);
}
```

D9

```
int[][] m = {
    {1, 2, 3},
    {4, 5, 6}
};
System.out.println(m[0][1]); // 2
```

D10

```
int voto = s.nextInt();

if (voto < 6) {
    System.out.println("insufficiente");
} else if (voto <= 7) {
    System.out.println("sufficiente");
} else if (voto <= 9) {
    System.out.println("buono");
} else if (voto == 10) {
    System.out.println("ottimo");
}
```

D11

```
int scelta = s.nextInt();

switch (scelta) {
    case 1:
        System.out.println("uno");
        break;
    case 2:
        System.out.println("due");
        break;
    case 3:
        System.out.println("tre");
        break;
    default:
        System.out.println("altro");
}
```

D12

```
try {
    int a = 10;
    int b = 0;
    System.out.println(a / b);
} catch (Exception e) {
    System.out.println("errore");
} finally {
    System.out.println("fine");
}
```

D13 - BONUS

```
Scanner s = new Scanner(System.in);
int[] numeri = new int[5];

for (int i = 0; i < numeri.length; i++) {
    numeri[i] = s.nextInt();
}

int maggiore = numeri[0];
for (int i = 1; i < numeri.length; i++) {
    if (numeri[i] > maggiore) {
        maggiore = numeri[i];
```

```
    }
}
```

```
System.out.println("Maggiore: " + maggiore);
```

16. Checklist finale del 27

- ☐ Scrivo il main completo senza guardare.
- ☐ Spiego .java → .class → JVM.
- ☐ Creo uno Scanner e leggo String/int/double.
- ☐ Distinguo = da == e && da ||.
- ☐ Spiego ++x e x++ con un esempio.
- ☐ Scrivo la formula Math.random(min,max) a memoria.
- ☐ Scrivo if/else, switch, while, do-while e for senza modello.
- ☐ Scorro un array con for e foreach.
- ☐ So leggere un elemento di una matrice con [riga][colonna].
- ☐ Spiego try/catch/finally e BufferedReader.

> TRICK: Se il 26 sera riesci a fare almeno 8/10 punti senza appunti e gli esercizi base senza copiare, il ripasso del 27 deve essere breve e mirato.